

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task No.: 5

Student Name: T. Akshaya

Roll No.: A24126552122

Date:

1. TITLE

Interactive Product Calculator — Dynamic Calculation using JavaScript Events and DOM Manipulation

2. AIM

To design a webpage that calculates the total price of a product dynamically using JavaScript functions, variables, and event listeners, and to update the result on the page without refreshing it.

3. OBJECTIVE

- To create input fields for product name, quantity, and price using HTML.
 - To use JavaScript functions to perform calculations based on user input.
 - To attach event listeners to input fields so the result updates in real time.
 - To manipulate the DOM to display the calculated total without reloading the page.
 - To validate user input and display appropriate error messages for invalid values.
-

4. THEORY

JavaScript enables web pages to respond to user actions dynamically without reloading. The Document Object Model (DOM) represents the HTML page as a tree of objects that JavaScript can read and modify at runtime using methods such as `getElementById()` and properties such as `textContent`.

Event listeners, attached using `addEventListener()`, allow a function to run automatically whenever a specific event occurs — such as the "input" event, which fires every time the value of a text or number field changes. By combining event listeners with a calculation function, a webpage can

recompute and redisplay a result instantly as the user types, giving the page an interactive, app-like feel.

Basic input validation, using `isNaN()` and comparison operators, ensures that only valid numeric values are used in the calculation, and that the user is warned when the entered quantity or price is missing or invalid.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file and set up the basic HTML5 structure with a title "Interactive Product Calculator".
 2. Add input fields for Product Name (text), Quantity (number), and Price per Unit (number) inside a container.
 3. Add a result section with elements to display the product name and the calculated total price.
 4. In JavaScript, store references to all input fields and output elements using `document.getElementById()`.
 5. Define a function `calculateTotal()` that reads the current values of quantity and price using `parseFloat()`.
 6. Inside the function, validate the inputs using `isNaN()` and check that quantity and price are not negative; display an error message if invalid.
 7. If the inputs are valid, calculate the total as $\text{quantity} \times \text{price}$ and update the result section using `textContent`.
 8. Attach an "input" event listener to each of the three fields, calling `calculateTotal()` whenever a value changes.
 9. Save the HTML file and open it in a web browser to verify that the total updates live as values are entered.
 10. Test the calculator with valid values, empty fields, and invalid (negative) values, and record the results as test cases.
-

6. TASK DESCRIPTION

A webpage titled "Interactive Product Calculator" was created, covering the following parts:

Part A: Structure

- Created input fields for Product Name, Quantity, and Price per Unit.
- Added a styled result box to display the product name and total price.

Part B: JavaScript Logic

- Used `document.getElementById()` to reference all input and output elements.
- Wrote a `calculateTotal()` function that multiplies quantity and price to compute the total.
- Used `parseFloat()` and `isNaN()` to validate numeric input and handle empty or negative values.

Part C: Event Handling and Dynamic Update

- Attached "input" event listeners to the quantity, price, and product name fields.
 - Updated the DOM using textContent so the total price recalculates instantly without a page refresh.
 - Displayed a validation message when the quantity or price entered was missing or invalid.
-

7. PROGRAM

Task_5.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Interactive Product Calculator</title>
<style>
  body {
    font-family: Arial, sans-serif;
    background-color: #f4f6f8;
    display: flex;
    justify-content: center;
    padding: 40px;
  }
  .calculator-box {
    background-color: #ffffff;
    border: 1px solid #ccc;
    border-radius: 8px;
    padding: 25px 35px;
    width: 350px;
    box-shadow: 2px 2px 8px rgba(0,0,0,0.15);
  }
  h2 {
    text-align: center;
    color: #2c3e50;
  }
  label {
    display: block;
    margin-top: 15px;
    font-weight: bold;
    color: #333;
  }
  input[type="text"],
  input[type="number"] {
    width: 100%;
    padding: 8px;
```

```
margin-top: 5px;
border: 1px solid #aaa;
border-radius: 4px;
box-sizing: border-box;
}
```

```
.result-box {
  margin-top: 25px;
  padding: 15px;
  background-color: #eafaf1;
  border: 1px solid #2ecc71;
  border-radius: 6px;
  text-align: center;
}
.result-box h3 {
  margin: 0;
  color: #27ae60;
}
#error {
  color: #e74c3c;
  font-size: 14px;
  margin-top: 8px;
  text-align: center;
}
</style>
</head>
<body>

<div class="calculator-box">
  <h2>Interactive Product Calculator</h2>

  <label for="productName">Product Name:</label>
  <input type="text" id="productName" placeholder="Enter product name">

  <label for="quantity">Quantity:</label>
  <input type="number" id="quantity" placeholder="Enter quantity" min="0">

  <label for="price">Price per Unit (₹):</label>
  <input type="number" id="price" placeholder="Enter price" min="0" step="0.01">

  <div class="result-box">
    <p id="productDisplay">Product: --</p>
    <h3 id="totalPrice">Total Price: ₹0.00</h3>
  </div>
  <div id="error"></div>
</div>
```

```
<script>
  // Get references to the input fields and output elements
  const productNameInput = document.getElementById("productName");
  const quantityInput = document.getElementById("quantity");
  const priceInput = document.getElementById("price");
  const totalPriceDisplay = document.getElementById("totalPrice");
  const productDisplayEl = document.getElementById("productDisplay");
  const errorDisplay = document.getElementById("error");

  // Function to calculate and display the total price
  function calculateTotal() {
    const name = productNameInput.value.trim();
    const quantity = parseFloat(quantityInput.value);
    const price = parseFloat(priceInput.value);

    // Validation: check for empty or invalid numeric inputs
    if (isNaN(quantity) || isNaN(price) || quantity < 0 || price < 0) {
      totalPriceDisplay.textContent = "Total Price: ₹0.00";
      errorDisplay.textContent = "Please enter valid quantity and price.";
      productDisplayEl.textContent = "Product: " + (name || "--");
      return;
    }

    errorDisplay.textContent = "";
    const total = quantity * price;

    productDisplayEl.textContent = "Product: " + (name || "--");
    totalPriceDisplay.textContent = "Total Price: ₹" + total.toFixed(2);
  }

  // Event listeners: recalculate whenever the user types/changes a value
  productNameInput.addEventListener("input", calculateTotal);
  quantityInput.addEventListener("input", calculateTotal);
  priceInput.addEventListener("input", calculateTotal);
</script>

</body>
</html>
```

8. CODE EXPLANATION

Tag / Method	Purpose
<input type="text"> / "number"	Collects the product name, quantity, and price from the user.
document.getElementById()	Retrieves references to the input fields and result elements in the DOM.
addEventListener("input", ...)	Fires the calculateTotal() function immediately whenever the user types or changes a value, without needing a button click.
parseFloat()	Converts the text entered in the quantity and price fields into numeric values for calculation.
isNaN() validation	Checks for empty, non-numeric, or negative entries and displays an error message instead of an incorrect total.
textContent	Updates the product name and total price on the page dynamically (DOM manipulation) without reloading it.
.toFixed(2)	Formats the total price to two decimal places for a proper currency display.

9. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Load the webpage with all fields empty	Total Price shows ₹0.00 and no error message	Displayed ₹0.00, no error shown	Pass
TC-02	Enter Product Name = "Wireless Mouse", Quantity = 3, Price = 499.50	Total Price updates instantly to ₹1498.50 without refresh	Updated correctly to ₹1498.50	Pass
TC-03	Enter a negative value in Quantity (e.g., -2) with Product Name and Price filled	Validation error message is displayed and total is not calculated	Error shown: "Please enter valid quantity and price."	Pass
TC-04	Change Price to 650 while Quantity (3) is already filled	Total Price recalculates	Recalculated live to ₹1950.00	Pass

Test Case	Action	Expected Result	Actual Result	Status
		automatically on every keystroke		
TC-05	Clear the Quantity field after a valid entry (Price still filled)	Total resets to ₹0.00 and validation message appears	Displayed ₹0.00 with validation message	Pass

10. OUTPUT

Output 1 (TC-01): Initial Page Load (Default State)

Interactive Product Calculator

Product Name:

Enter product name

Quantity:

Enter quantity

Price per Unit (₹):

Enter price

Product: --

Total Price: ₹0.00

Fig 1 — Calculator on page load, before any values are entered

Output 2 (TC-02): Live Calculation After Entering Valid Values

Interactive Product Calculator

Product Name:
Wireless Mouse

Quantity:
3

Price per Unit (₹):
499.50

Product: Wireless Mouse
Total Price: ₹1498.50

Fig 2 — Total price updates instantly as Quantity and Price are entered ($3 \times ₹499.50 = ₹1498.50$)

Output 3 (TC-03): Validation on Invalid (Negative) Input

Interactive Product Calculator

Product Name:
Bluetooth Speaker

Quantity:
-2

Price per Unit (₹):
899

Product: Bluetooth Speaker
Total Price: ₹0.00

Please enter valid quantity and price.

Fig 3 — Error message shown when Quantity is negative; total is not calculated

Output 4 (TC-04): Total Recalculates When Price Is Changed

Interactive Product Calculator

Product Name:
Wireless Mouse

Quantity:
3

Price per Unit (₹):
650

Product: Wireless Mouse
Total Price: ₹1950.00

Fig 4 — Total price updates live when Price is changed to 650 ($3 \times ₹650 = ₹1950.00$)

Output 5 (TC-05): Total Resets When Quantity Is Cleared

Interactive Product Calculator

Product Name:
Wireless Mouse

Quantity:
Enter quantity

Price per Unit (₹):
650

Product: Wireless Mouse
Total Price: ₹0.00

Please enter valid quantity and price.

Fig 5 — Total resets to ₹0.00 and validation message appears when Quantity is cleared

11. RESULT

Thus, a webpage titled "Interactive Product Calculator" was successfully designed and developed using HTML, CSS, and JavaScript. JavaScript functions, DOM manipulation, and "input" event listeners were used to dynamically calculate and display the total price whenever the quantity or price changed, without refreshing the page. The page was tested with valid and invalid inputs in a web browser, and all features were verified to work correctly.